

Introduction

Picture an array of 1,500 numbers. Someone asks you: pick any three elements (repeats allowed), XOR them together, and count how many distinct results are possible. Your gut says the answer could be in the millions — there are billions of ways to pick three indices from 1,500 elements. But the real answer is capped at a few thousand, and once you see why, you'll never look at a "bounded values" constraint the same way again.

This is exactly the kind of problem that shows up in interviews at companies like Google and Bloomberg, not because XOR itself is exotic, but because it tests something more valuable than knowing an operator: can you notice when a *value* constraint (rather than an *array length* constraint) changes your entire complexity strategy? XOR also isn't just an interview toy — it's the backbone of checksums, parity checks, and error detection in real systems. Learning to reason about it here pays off well beyond LeetCode.

By the end of this article, you'll know how to go from a naive triple-loop to a clean, fast solution using nothing more exotic than Python sets — and you'll understand *why* it works, not just *that* it works.

Problem Overview

You're given an array of integers. Consider every triplet of indices (i, j, k) such that $i \leq j \leq k$ — meaning an index can repeat, so the same element can be used more than once in a triplet, and order doesn't matter beyond this non-decreasing constraint.

For each such triplet, compute $\text{nums}[i] \oplus \text{nums}[j] \oplus \text{nums}[k]$. Your task is to count how many **distinct** XOR values can be produced across all valid triplets.

Two ideas anchor this problem:

- A **set** never stores duplicates. Insert the same value twice, and it only shows up once — like a guest list where writing a name that's already there does nothing.
- **XOR ($\oplus$)** compares two numbers bit by bit. Matching bits produce 0 , differing bits produce 1 . A useful shortcut: $x \oplus x = 0$, and $x \oplus 0 = x$.

Example

Input: `nums = [2, 2]`

Output: 1

With only the value `2` available (even repeated), every possible triplet is `2 ^ 2 ^ 2`. Since `2 ^ 2 = 0` and `0 ^ 2 = 2`, every single triplet evaluates to `2`. There's exactly one distinct result.

Input: `nums = [1, 2, 3]`

Output: 4

Here the unique triplet XORs include values like `1^1^1=1`, `1^2^3=0`, `2^2^3=3`, `1^1^2=2`, and more. Working through all combinations, the distinct results that show up are `{0, 1, 2, 3}` — four unique values.

Intuition

The instinct every beginner reaches for is to just try all three nested loops over `i, j, k` and collect results in a set. That's completely reasonable as a first move — it's simple, obviously correct, and easy to verify by hand.

The problem is scale. With 1,500 elements, the number of valid `(i, j, k)` triplets is on the order of `n^3 / 6`, which is over half a billion — even more if you don't bound `i <= j <= k`. That's too slow to run in any reasonable time limit.

So instead of asking "what are all the triplets I can form?", flip the question: "what are all the *values* I can reach by XOR-ing one, two, or three numbers together?" That reframing is the entire trick. You build up the set of reachable values in stages:

1. Start with the unique values in the array (numbers reachable using just one element).
2. XOR every pair of unique values together to get every value reachable using two elements.
3. XOR every value from step 2 with every unique value again to get every value reachable using three elements.

Because a set silently discards duplicates at each stage, you never do wasted work twice, and the final set size is your answer.

The reason this stays fast — and this is the part that makes everything click — is that XOR never produces a result needing more bits than its inputs. If every number in the array is smaller than 1,500, each one fits in 11 bits, so every possible XOR of them is smaller than `2^11 = 2048`. That means the "reachable in two XORs" set can *never* contain more than 2048 values, no matter how large the input

array grows. That fixed ceiling is what turns an exponential-feeling problem into something that scales gracefully.

Brute Force Solution

Idea: Enumerate every triplet (i, j, k) with $i \leq j \leq k$ directly, compute the XOR, and throw each result into a set.

Algorithm: 1. Initialize an empty set `results`. 2. For each i from 0 to $n-1$: - For each j from i to $n-1$: - For each k from j to $n-1$: - Add $\text{nums}[i] \oplus \text{nums}[j] \oplus \text{nums}[k]$ to `results`. 3. Return `len(results)`.

Advantages: Trivial to write, trivial to verify by hand, and perfectly fine for small inputs or one-off scripts.

Disadvantages: Cubic time makes it unusable once n approaches the problem's real limits.

Complexity: Time $O(n^3)$, space $O(2048)$ bounded by the distinct XOR values stored (though the array itself could theoretically hold up to n^3 before deduplication, in practice the set caps at the same bound as the optimal solution).

Optimal Solution

Instead of looping over index triplets, build up XOR-reachability in stages:

Stage	What it represents	Set size bound
<code>uniq</code>	Values reachable with 1 pick	up to n , but really up to 2048
<code>xor2</code>	Values reachable with 2 picks (pairwise XOR of <code>uniq</code>)	up to 2048
<code>xor3</code>	Values reachable with 3 picks (XOR of <code>xor2</code> with <code>uniq</code>)	up to 2048

Step 1 — Deduplicate. Convert `nums` to a set. A repeated value can never produce a new XOR result that a single copy couldn't already produce ($x \oplus x = 0$ just cancels out), so duplicates are pure wasted work.

Step 2 — Build all pairwise XORs. For every pair of unique values (including a value paired with itself), XOR them and store the result in `xor2`. This is $O(u^2)$ where $u = \text{len}(\text{uniq})$, but since u is capped at roughly 2048 for this problem's constraints, this step stays fast even at the largest allowed n .

Step 3 — Extend to triples. For every value in `xor2`, XOR it against every value in `uniq` again. Store all results in `xor3`. This simulates three-way XOR without ever forming a triple explicitly — you're really just doing "reachable in 2" XOR "reachable in 1" one more time.

Step 4 — Return the count. `len(xor3)` is your answer.

Python Code

```
def unique_xor_triplets(nums: list[int]) -> int:
    """Count distinct XOR values from all triplets (i <= j <= k, repeats allowed)."""
    uniq = set(nums)

    # Every value reachable by XOR-ing two elements (including a value with itself)
    xor2 = {a ^ b for a in uniq for b in uniq}

    # Every value reachable by XOR-ing that result with one more element
    xor3 = {x ^ a for x in xor2 for a in uniq}

    return len(xor3)
```

Code Walkthrough

- `uniq = set(nums)` — deduplicates the input. Since $a \oplus a = 0$ adds nothing new, keeping duplicates around would only waste cycles in the next two steps.
- `xor2 = {a ^ b for a in uniq for b in uniq}` — a set comprehension that XORs every ordered pair of unique values (including `a` with itself). Because it's a set, any duplicate result is automatically dropped.
- `xor3 = {x ^ a for x in xor2 for a in uniq}` — takes every "reachable in two" value and XORs it against every unique original value, producing every "reachable in three" value.
- `return len(xor3)` — the final answer is simply how many distinct values survived into the last set.

Dry Run

Take `nums = [1, 2, 3]`.

1. `uniq = {1, 2, 3}`
2. Build `xor2` :
3. $1 \oplus 1 = 0$, $1 \oplus 2 = 3$, $1 \oplus 3 = 2$

4. $2^1=3, 2^2=0, 2^3=1$
5. $3^1=2, 3^2=1, 3^3=0$
6. `xor2 = {0, 1, 2, 3}`
7. Build `xor3` (XOR each value in `xor2` with each in `uniq`):
8. $0^1=1, 0^2=2, 0^3=3$
9. $1^1=0, 1^2=3, 1^3=2$
10. $2^1=3, 2^2=0, 2^3=1$
11. $3^1=2, 3^2=1, 3^3=0$
12. `xor3 = {0, 1, 2, 3}`
13. `len(xor3) = 4` — matches the expected output from the earlier example.

Complexity Analysis

- **Time:** $O(n)$ to dedupe, plus $O(u^2)$ to build `xor2`, plus $O(u^2)$ to build `xor3`, where $u = \min(n, 2048)$ for this problem's value limits. Since u is capped by the bit-width of the value domain rather than growing with n , the practical cost is $O(2048^2)$ in the worst case — a fixed ceiling, not something that grows with input size.
- **Space:** $O(u)$ for each set, bounded by the same 2048 ceiling.
- **Why this is correct:** XOR of values under 2^k can never require more than k bits, so the reachable set at every stage is bounded by 2^k regardless of how many elements feed into it. This is the same reasoning used in bitmask DP: when the *value* domain is small, the *state space* is small, even if the input is large.

Alternative Solutions

You could try building all three stages in a single loop with manual bit tricks, but it won't beat the set-based closure approach in clarity or performance — Python's set operations are already implemented in C and handle deduplication efficiently. Another alternative is precomputing pairwise XORs into a sorted array and binary-searching, but that adds complexity without any real speed benefit here, since the set comprehension is already bounded by the same 2048 ceiling.

For small inputs (a short list in a quick script), the brute-force triple loop is a perfectly reasonable "alternative" — it's easier to read and just as fast in practice when n is small.

Edge Cases

- **Empty array:** Not valid per constraints, but if encountered, `uniq` would be empty and the function should return `0`.
- **Single element:** `uniq` has one value, `xor2` has one value (`x^x=0` ... wait, actually one value since `{x^x}` collapses to `{0}` only if there's just one unique value — trace carefully), and `xor3` ends up with exactly one value, correctly returning `1`.
- **All duplicates:** Collapses to the same case as a single unique element — always returns `1`.
- **Maximum array size (1,500 elements):** The middle and final sets never exceed `2048` entries regardless of how large `n` grows, so performance stays flat.
- **All distinct, maximum value range:** Stress-tests the `2048` bound directly — a good sanity check that your solution doesn't silently blow past the expected ceiling.

Common Mistakes

1. **Skipping deduplication** — leads to unnecessary repeated work in the pairwise XOR step without changing correctness, just wasting time.
2. **Forgetting to XOR a value with itself** — self-pairing (`a^a=0`) matters because it's how `0` enters the reachable set.
3. **Building triplets directly with three nested loops** — technically correct but far too slow for the upper constraint.
4. **Assuming the answer scales with n^3** — missing the fact that the value range, not the array length, bounds the answer once values are small.
5. **Using a list instead of a set** for intermediate results — this reintroduces duplicate work and inflates both time and space unnecessarily.

Interview Questions

- What if the problem asked for k-tuples instead of triplets? (Answer: repeat the closure step `k-1` times.)
- What if the array length were bounded but the values were unbounded? (Answer: the bounded-reachable-set trick breaks down; you'd need a different bound or accept `O(n^3)`.)
- Can you prove the upper bound on the number of distinct XOR values without running the code?
- Why is deduplication safe rather than just an optimization?

Similar Problems

- **LeetCode: Number of Unique XOR Triplets I** — the simpler version of this exact problem, if it exists at smaller constraints.
- **LeetCode: XOR Queries of a Subarray** — reinforces prefix-XOR reasoning.
- **LeetCode: Maximum XOR of Two Numbers in an Array** — uses a similar bit-level closure idea via a trie.
- **LeetCode: Single Number** — foundational XOR self-cancellation intuition ($x \oplus x = 0$).

These are related because they all lean on the same core property: XOR results are bounded by the bit-width of the inputs, and reasoning about reachable values (rather than brute-force enumeration) is the key unlock.

Key Takeaways

When a problem bounds the *values* instead of the *count* of elements, ask whether the reachable result space is what's actually capped. Build closures over that bounded space in stages rather than enumerating every combination of indices. Dedup early, let sets do the heavy lifting, and always sanity-check your solution against the smallest possible input before trusting it on the largest.

Watch the Video

For the full walkthrough with a live pause-and-think moment and a quick quiz to test your understanding, watch the video here: {LINK}

About the Series

This breakdown is part of the Daily Python LeetCode series, where each video takes one interview-relevant problem and builds the solution from first principles — brute force first, then the optimal approach, explained at beginner, intermediate, and advanced levels so there's something useful no matter where you're starting from.

Call To Action

If this helped clarify how bounded value ranges change your complexity thinking, hit like on the video, subscribe so you don't miss tomorrow's problem, and subscribe to the Substack for the full written breakdown delivered straight to your inbox. Drop a comment with your own approach or a problem you'd like covered next, and share this with anyone prepping for interviews.

Quick Review

You see 'count distinct XOR results from triples' with $n \leq 1500$ — what's the pattern?

Bounded-value dedup: build a set of pairwise XORs first, then XOR that set with each element into a final set.

Time complexity of the staged-set approach for triplet XOR?

$O(n^2)$ to build pairwise XOR set, plus $O(n \cdot |\text{set}|)$ to combine with third element — far better than brute-force $O(n^3)$.

Why does space stay small despite n up to 1500?

XOR results are capped by value bit-width (e.g. $\leq 2^{16}$), so the set can't grow past that regardless of n .

Common bug when building the XOR set in stages?

Forgetting $i < j < k$ ordering doesn't matter for XOR (commutative), but reusing the same index twice does — must pick distinct triple positions.

Why not brute-force all n^3 triples here?

Looks fine at first since n is small, but $1500^3 \approx 3.4$ billion ops — too slow; the trick exploits limited output range instead.

Set + XOR combo — when else does this pattern apply?

Any 'count distinct combined values' problem where the combining op is associative/commutative and output space is bounded, not input size.